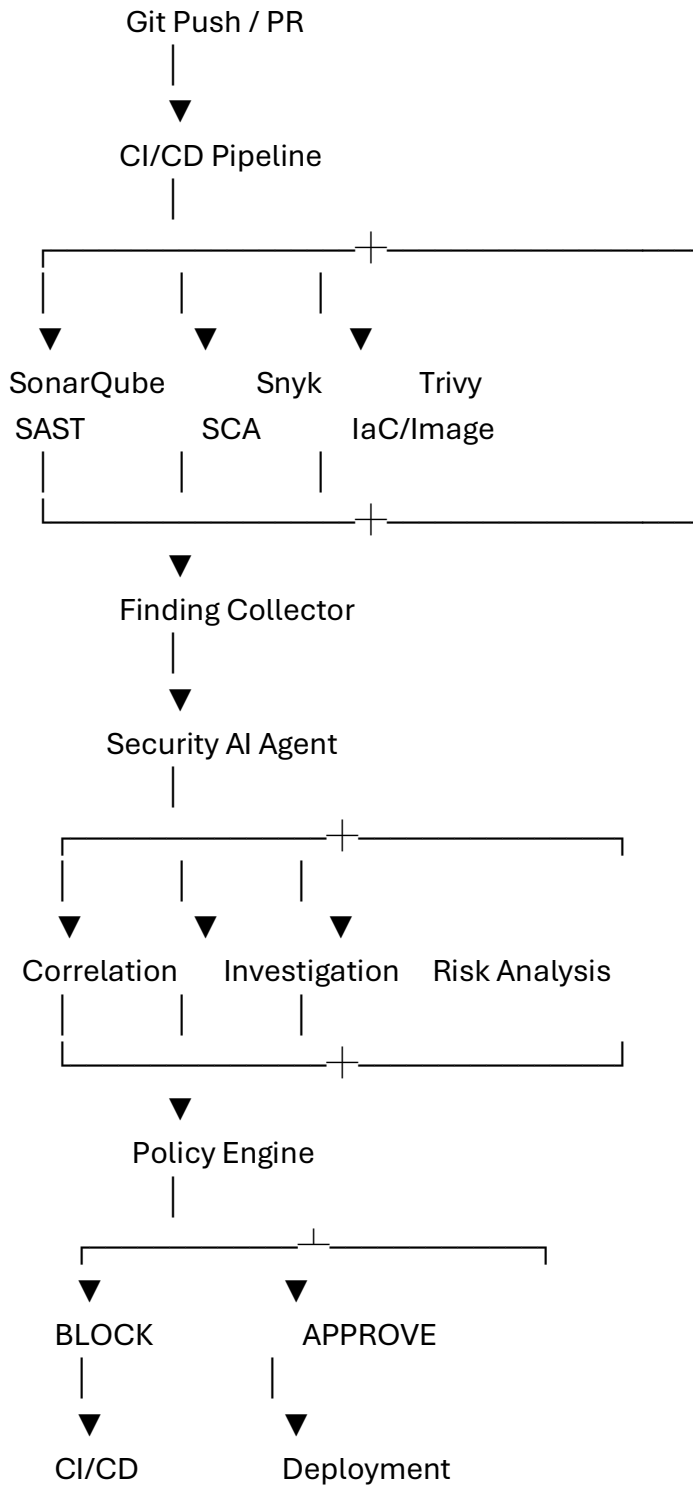


The architecture should look like this:



1. Give the agent tools, not direct access

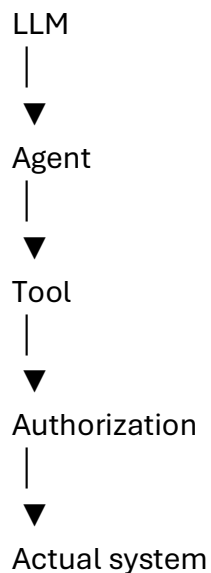
Your AI agent should have controlled tools such as:

```
get_sonarqube_findings()
get_snyk_findings()
get_trivy_findings()
get_pipeline_status()
get_git_diff()
get_commit_details()
get_sbom()
get_asset_context()
get_cve_details()
create_security_ticket()
comment_on_pr()
request_approval()
```

Avoid giving it:

```
execute_shell()
kubectl_anything()
delete_production()
run_arbitrary_sql()
```

The distinction is important:



rather than:

LLM —————▶ Production

2. Create a Pipeline Security Agent

For example, call it:

Pipeline Security Agent

Its job is:

1. Monitor pipeline events.
2. Collect SonarQube/Snyk/Trivy results.
3. Correlate findings.
4. Investigate suspicious findings.
5. Determine contextual risk.
6. Recommend remediation.
7. Enforce predefined policy.
8. Notify developers/security teams.
9. Maintain an audit trail.

The agent receives something like:

```
{  
  "pipeline_id": "48291",  
  "repository": "payment-api",  
  "branch": "feature/login",  
  "commit": "a81f92",  
  "environment": "staging"  
}
```

Then it calls its tools.

3. Agent calls SonarQube

The agent might execute:

```
get_sonarqube_findings(  
  project="payment-api",  
  pipeline="48291"  
)
```

Result:

```
{  
  "findings": [  
    {  
      "type": "SQL Injection",  
      "severity": "HIGH",  
      "file": "src/login.py",  
      "line": 142  
    }  
  ]  
}
```

The agent doesn't immediately block the deployment.

It asks:

Is this exploitable? Is the vulnerable code reachable? Is this a new finding?

4. Agent calls Snyk

Then:

```
get_snyk_findings(  
  project="payment-api",  
  pipeline="48291"  
)
```

Suppose it finds:

requests 2.x

CVE-XXXX

CVSS 9.8

Fix available: 2.x.x

Now the agent has two independent sources:

SonarQube

↓

Application vulnerability

Snyk

↓

Dependency vulnerability

5. Agent calls Trivy

Next:

```
get_trivy_findings(  
    image="payment-api:a81f92"  
)
```

Result:

openssl

CVE-XXXX

CRITICAL

libssl

CVE-YYYY

HIGH

And possibly:

Kubernetes deployment
privileged: true

Now the agent has:

Application
+
Dependencies
+
Container
+
Infrastructure

6. The interesting part: correlation

This is where an AI agent becomes useful.

Imagine:

SonarQube:
SQL injection — HIGH

Snyk:
No related vulnerable dependency

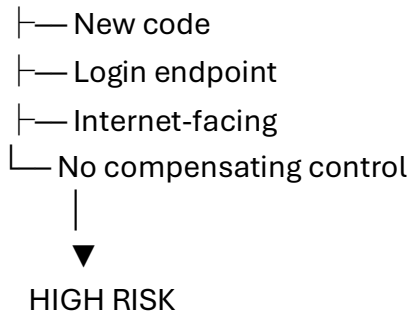
Trivy:
No relevant CVE

Git:
Changed login API

Application:
Internet-facing

The agent can correlate the evidence:

SQL Injection
|



Compare that with:

SonarQube:

SQL injection — HIGH

Git:

Finding exists in legacy test code

Application:

Not deployed

Endpoint:

Not reachable

The technical severity from SonarQube may still be HIGH, but the **deployment context** is different.

This is why you shouldn't blindly use:

CVSS = deployment decision

Instead:

Severity

+

Exploitability

+

Exposure

+

Asset criticality

+

Reachability

+
Existing controls
↓
Risk context

7. Give the agent access to Git

This is extremely useful.

Add:

```
get_git_diff()  
get_commit_details()  
get_file_content()
```

Then the agent can investigate:

SonarQube:
SQL Injection at login.py:142

Agent:

```
get_git_diff(commit=a81f92)
```

It discovers:

```
+ query = "SELECT * FROM users WHERE username='" + username + "'"
```

Now it has concrete evidence that the finding was introduced by the current PR.

It can produce:

Finding:
SQL Injection

Introduced by:
Commit a81f92

Location:

src/login.py:142

Affected component:
Authentication API

Exposure:
Internet-facing

Recommendation:
Use parameterized query / prepared statement.

Pipeline action:
BLOCK

8. Add RAG

This is where your previous RAG/Agentic SOC knowledge fits.

Create a security knowledge base containing:

OWASP
CWE
Internal secure coding standards
Company security policies
Incident response playbooks
Exception policies
Architecture standards
Previous security findings
Remediation guides

Then:

Agent
|
▼
RAG
|

- └─ CWE-89
- └─ OWASP SQL Injection
- └─ Internal policy
- └─ Secure coding standard

The LLM can now explain:

Why is this dangerous?

How can it be fixed?

Does company policy require blocking?

What evidence supports the finding?

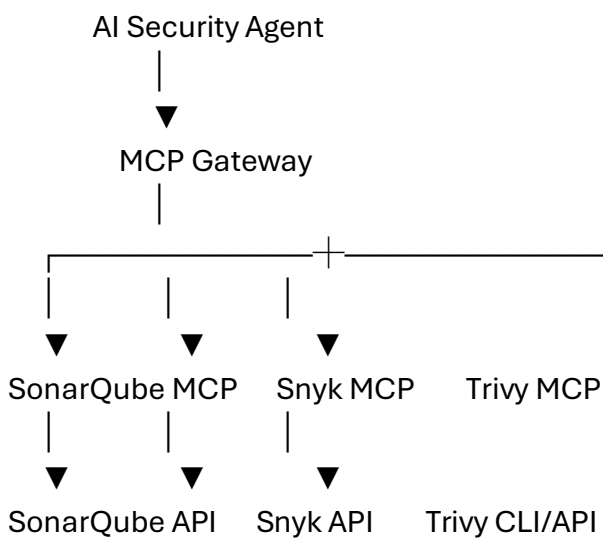
Important:

RAG content should be treated as evidence, not executable instructions.

9. Use MCP for the tools

Since you're specifically interested in MCP, MCP is a natural way to expose these systems.

You could build:



Additional MCP tools:

Git MCP
CI/CD MCP
Jira MCP
Slack/Teams MCP
CMDB MCP
Kubernetes MCP
Threat Intelligence MCP

For example:

get_sonarqube_issue
get_snyk_vulnerability
get_trivy_image_report
get_git_diff
get_pipeline_logs
create_jira_ticket
comment_pull_request

10. Don't let the LLM decide authorization

This is probably the most important architectural principle.

Don't do:

LLM

↓

"Critical vulnerability"

↓

LLM executes

↓

Block production

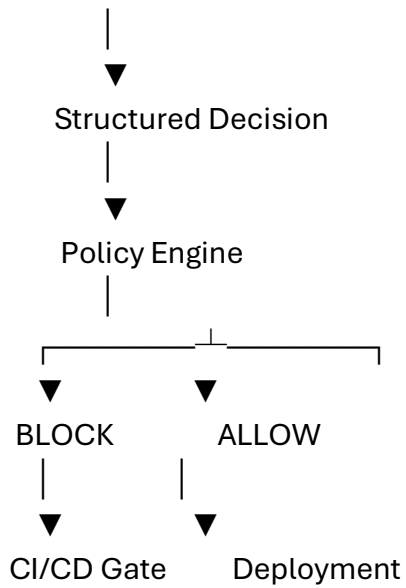
Instead:

AI Agent

|



Risk Assessment



For example:

```
{
  "decision": "BLOCK",
  "reason": "Critical vulnerability",
  "confidence": 0.94,
  "evidence": [
    "SonarQube finding 1234",
    "Git commit a81f92",
    "Internet-facing asset"
  ]
}
```

Then the **policy engine**, not the LLM, decides:

```
IF
  severity == CRITICAL
AND
  finding_is_new == true
AND
  internet_exposed == true

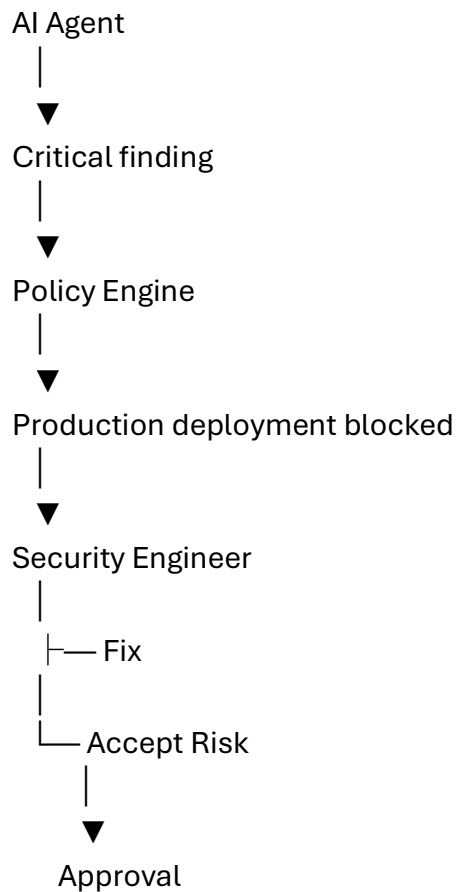
THEN
  BLOCK
```

This makes the system deterministic and auditable.

11. Human-in-the-loop

For exceptions, use human approval.

Example:



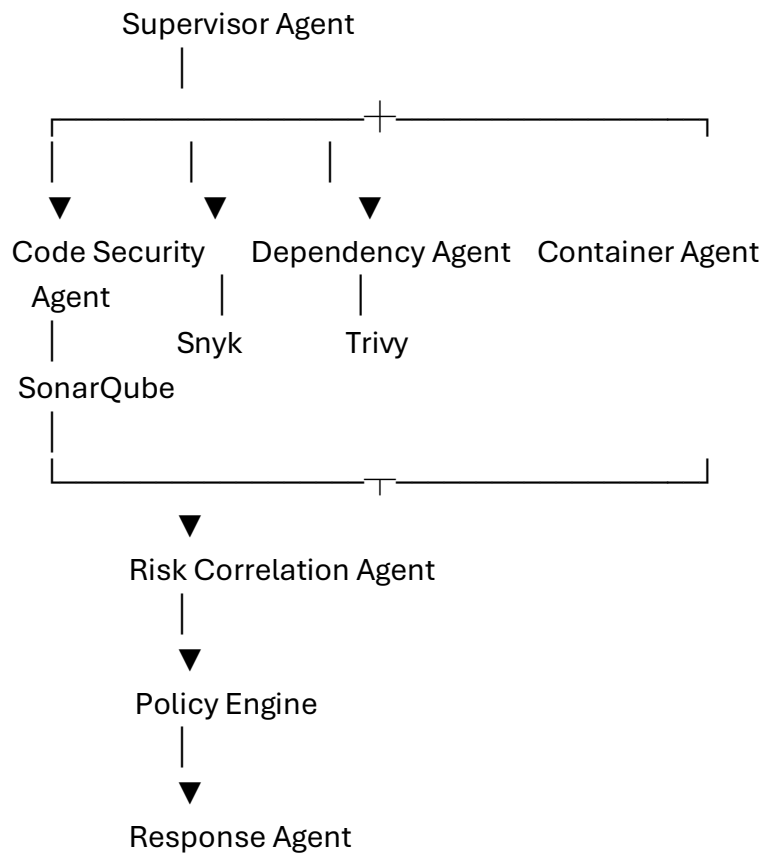
The approval should be recorded:

```
{
  "finding": "CVE-XXXX",
  "decision": "RISK_ACCEPTED",
  "approved_by": "security-engineer",
  "expires": "2026-12-31",
```

```
"business_justification": "..."  
}
```

12. Agent-to-agent architecture

You can make this more advanced by using multiple specialized agents.



Code Security Agent

Analyzes:

SonarQube
Git diff
CWE
OWASP

Dependency Agent

Analyzes:

Snyk
SBOM
CVE
CVSS
EPSS
package reachability

Container Agent

Analyzes:

Trivy
Dockerfile
Kubernetes
SBOM
container configuration

Risk Correlation Agent

Combines everything.

Response Agent

Can:

comment PR
create Jira ticket
notify Teams/Slack
request approval
update CI status

High-impact changes should remain gated by authorization.

13. Example end-to-end investigation

Suppose developer pushes:

PR #482

Pipeline starts.

Step 1

Gitleaks

↓

PASS

Step 2

SonarQube

↓

HIGH: SQL Injection

Agent starts investigation.

Step 3

Git diff

↓

Finding introduced in current PR

Step 4

Snyk

↓

No related dependency vulnerability

Step 5

Trivy

↓

No relevant container vulnerability

Step 6

Agent checks asset inventory:

Application:
payment-api

Exposure:
Internet-facing

Criticality:
High

Step 7

Agent retrieves internal policy:

Internet-facing application
+
new HIGH SQL injection
=
deployment requires remediation

Step 8

Policy engine:

BLOCK

Step 9

Agent comments on PR:

Security Gate: FAILED

Finding:
SQL Injection

Location:

src/login.py:142

Source:

SonarQube

Introduced:

Current PR

Exposure:

Internet-facing

Required remediation:

Use parameterized SQL queries.

Pipeline:

BLOCKED

This is a genuinely useful AI-agent use case because the agent is doing **correlation and investigation**, rather than simply repeating scanner output.

14. What you should build as a practical project

Given your Agentic SOC focus, I'd build it in these stages:

Phase 1 — Deterministic pipeline

GitHub

↓

SonarQube

↓

Snyk

↓

Trivy

↓

Security Gate

Phase 2 — Security data layer

Add:

PostgreSQL

+

Vector DB

+

SBOM

+

CVE database

Phase 3 — AI agent

Use:

Python

+

LangGraph

+

LLM

Architecture:

Pipeline Event

↓

Supervisor

↓

Finding Collector

↓

Investigation Agent

↓

Risk Agent

↓

Policy Engine

Phase 4 — MCP

Create:

SonarQube MCP

Snyk MCP

Trivy MCP

GitHub MCP

Jira MCP

Phase 5 — Security controls

Add:

RBAC

OAuth/service identities

MCP authorization

tool allowlists

rate limiting

audit logs

prompt-injection defenses

structured outputs

human approval

Phase 6 — Agent evaluation

Attack your own system with:

Malicious PR

Prompt injection in source comments

Poisoned README

Malicious dependency

Fake CVE information

Poisoned RAG document

MCP tool poisoning

Unauthorized tool invocation

Privilege escalation

Then evaluate it against the **OWASP LLM/Agentic AI/MCP risk categories** you were asking about earlier.

The resulting project is essentially a **DevSecOps Agentic Security Platform**:

